

IT-Sicherheit

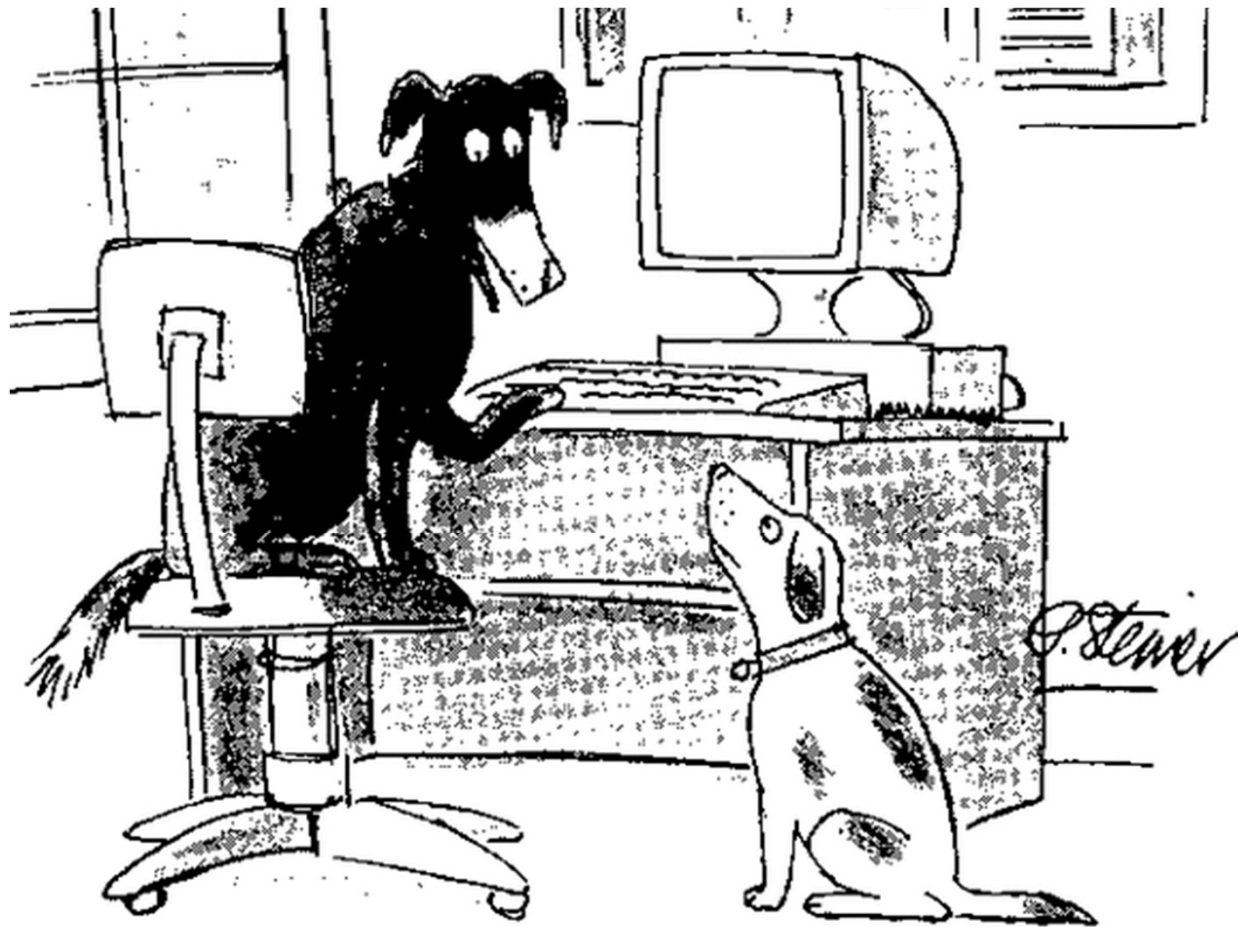
3. Authentifikationsverfahren

Prof. Dr. Tobias Straub
Masterprogramm Informatik

www.dhbw.de

Authentifikationsverfahren

- Gliederung:
 - Credentials, Multi-Factor Authentication
 - Biometrische Verfahren
 - Passwörter, Einmalpasswörter
 - Challenge-Response-Verfahren
 - Single-Sign-On (OpenID, Shibboleth)
 - Authentifikation und Autorisierung mittels SAML
- Literatur: [Eckert: Kap. 10, Kap. 13]



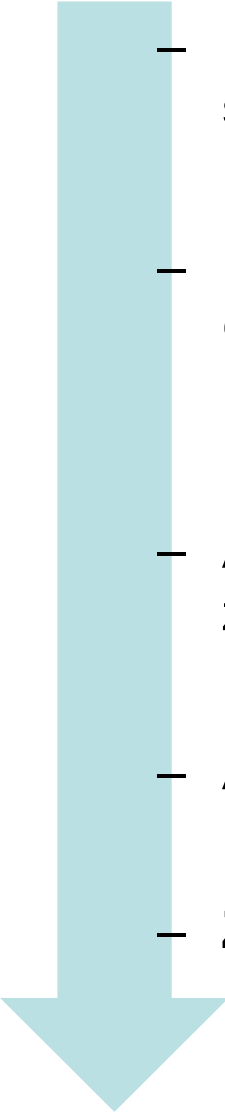
"On the Internet, nobody knows you're a dog."

[Peter Steiner, Published in The New Yorker July 5, 1993]

Einführung

- Anforderung zum Schutz gegen Maskierung und unautorisierten Zugriff:
 - Subjekte und (sicherheitsrelevante) Objekte müssen *eindeutig identifizierbar* sein
 - Subjekte müssen Identität nachweisen (sich authentifizieren) können, dazu sind *Credentials* (z.B. Username+Passwort, Zertifikat) vorzuweisen, Unterscheidung:
 - Authentifikation durch *Wissen*
 - Authentifikation durch *physischen Besitz*
 - Authentifikation durch *biometrisches Merkmal*
 - *2-/3-Faktor-Auth.:* Kombination mehrerer Merkmale, etwa Chipkarte + PIN
- oftmals genügt Authentifizierung zu Beginn einer Sitzung (etwa System-Login)
- *erneute Authentifizierung für sicherheitsrelevante Aktionen* ratsam: z.B. Software-Installation, Passwort-Änderung, einzelne Überweisung bei Online-Banking
- *beidseitige Authentifikation:* auch System authentifiziert sich gegenüber Benutzern (nicht immer gegeben, dadurch Maskierungsangriffe möglich)

Begrifflichkeiten

- 
- **Identität:** Summe charakteristischer Merkmale, die Unterscheidung erlauben, statisch (z.B. Name, SteuerID) oder dynamisch (z.B. IP-Adresse)
 - **Identifizierung:** Prozess der Mitteilung der Identität eines Subjekts oder Objekts (z.B. eines Benutzers oder eines anderen Systems) an ein System. Mittel: z.B. Benutzername, physisches Objekt, biometrisches Merkmal
 - **Authentifizierung:** Überprüfung der (behaupteten) Identität durch System z.B. mittels Passwort, Challenge-Response-Verfahren (Public Key-Verf.)
 - **Autorisierung:** Zuweisung von Zugriffsrechten auf Objekte oder Ressourcen
 - **Zugriffskontrolle:** Überwachung des Zugriffs auf Objekte oder Ressourcen

Authentifikation durch Wissen

- "Klassiker": Benutzername zur Identifikation + Passwort zur Authentifikation
- mit bekannten Schwächen (mangelnde Usability, Wiederverwendung, Phishing, unsichere Speicherung)
- **Current Best Practices** (basierend auf NIST-Empfehlungen [[SP 800-63B](https://isacybersecurity.com/world-password-day-2021/)])
 - 1) More focus on increased password length over password complexity
 - 2) Employ a “deny list” of unacceptable passwords
 - 3) Never reuse passwords across sites and services
 - 4) Eliminate regularly-scheduled password resets
 - 5) Allow password “copy and paste”
 - 6) Time-outs on failed password attempts
 - 7) Don’t use password hints
 - 8) Use Multi-Factor Authentication
 - 9) Train staff on password best practices



Schwachstelle: Speicherung von Passwörtern auf Server

- Passwörter dürfen nicht im Klartext gespeichert sein, sondern nur deren Bild unter *geeigneter Einwegfunktion mit Salt* !
- **bereits 2012 !**

06.06.2012 15:16

« Vorige | Nächste



Alert! LinkedIn-Passwörter im Umlauf UPDATE



vorlesen / MP3-Download

In einschlägigen Internet-Foren kursiert derzeit eine Liste mit über 6 Millionen Passwort-Hashes, die angeblich von [LinkedIn](#) stammen. Die Passwörter werden derzeit gemeinschaftlich geknackt, für rund 300.000 wurde bereits der Klartext veröffentlicht. Es empfiehlt sich, jetzt sofort das eigene LinkedIn-Passwort zu ändern.

*... Erschreckend ist, dass selbst Passwörter wie "parikh093760239",
"a06v1203n08" oder "376417miata?" bereits geknackt wurden. ...
SHA1-Hashes offenbar **ohne Salt** erzeugt*

Wie gesagt: keine Speicherung von Passwörtern im Klartext

21 Facebook Stored Hundreds of Millions of User Passwords in Plain Text for Years

MAR 19

Hundreds of millions of Facebook users had their account passwords stored in plain text and searchable by thousands of Facebook employees — in some cases going back to 2012, KrebsOnSecurity has learned. Facebook says an ongoing investigation has so far found no indication that employees have abused access to this data.

The Facebook source said the investigation so far indicates between 200 million and 600 million Facebook users may have had their account passwords stored in plain text and searchable by more than 20,000 Facebook employees. The source said Facebook is still trying to determine how many passwords were exposed and for how long, but so far the inquiry has uncovered archives with plain text user passwords dating back to 2012.

! My Facebook insider said access logs showed some 2,000 engineers or developers made approximately nine million internal queries for data elements that contained plain text user passwords.

Quelle: <https://krebsonsecurity.com/2019/03/facebook-stored-hundreds-of-millions-of-user-passwords-in-plain-text-for-years/>

Angriffe auf Passwörter

- **Social Engineering:** oftmals *Phishing*, auch *Shoulder Surfing*
- **Online-Angriff auf Login-Dialog:**
 - nur interaktives Durchprobieren am Login-Formular möglich
 - ggf. automatisierbar durch Skript; Frequenz aber begrenzt, z.B. durch *Verzögerungsmechanismen* (Wartezeit, IP-, Account-Sperre, Captchas)
- **Offline-Angriff auf Passwort-Datenbank:**
 - Liste der Passwörter liegt vor und kann mit *Wörterbüchern*, *paralleler Rechenpower* und *Rainbow-Tables* leicht angegriffen werden
 - wird z.B. als Einwegfunktion MD5 ohne Salt
(`echo -n "qwerty" | openssl dgst`)
verwendet, so bietet dies wenig Schutz

dies war vielfach das Default-Verfahren, z.B. bei

- Typo3 [http://wiki.typo3.org/Faq/Reset_BE_password]
- Wordpress [http://codex.wordpress.org/Resetting_Your_Password]



<https://crackstation.net>



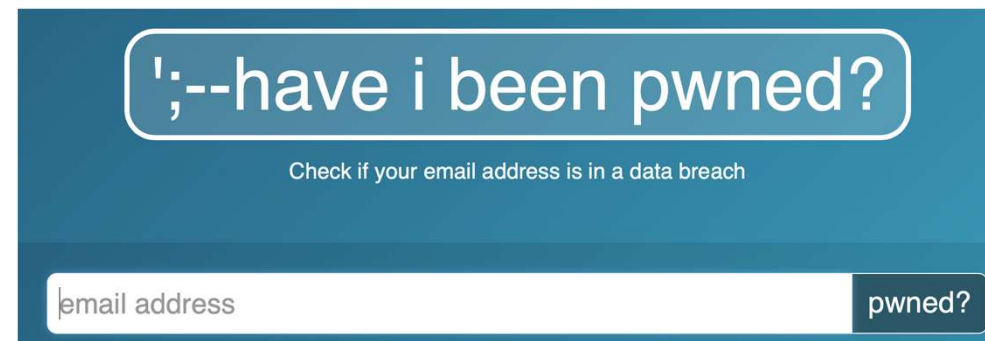
Selbsttest

- <https://haveibeenpwned.com>
sammelt Daten aus bekannten Leaks
 - ca. 17,5 Mrd kompromittierte Accounts
 - wird zB. von Safari verwendet (s.u.)
- aber: Passwort-Eingabe zur Prüfung ???

Funktionsweise:

- Passwort wird lokal im Browser per JavaScript gehasht (SHA-1)
- nur die ersten 5 Hex-Zeichen werden an Server gesendet, z.B.
<https://api.pwnedpasswords.com/range/640ab>
- Server antwortet mit Liste bekannter Hashes (und Anzahl der Vorkommen)
- Browser sucht lokal nach vollständigem Hash des Passworts (40 Hex-Zeichen)

Resultat: bei nicht-kompromittierten Passwort erhält Server nicht genügend Informationen, um Passwort zu brechen (20 Bit)



The screenshot shows the top part of the 'have i been pwned?' website. It has a dark blue header with the title in a white rounded rectangle. Below the title is a subtitle. At the bottom of the header is a search bar with a placeholder 'email address' and a 'pwned?' button.

';--have i been pwned?

Check if your email address is in a data breach

email address pwned?

Prüfung der Mail-Adresse



The screenshot shows a single input field for a password. It has a placeholder 'password' and a 'pwned?' button.

password pwned?

Prüfung des Passworts



The screenshot shows a portion of the Amazon.de website. It includes the Amazon logo, the domain 'amazon.de', and a 'Zuletzt geändert' timestamp. There is a 'Bearbeiten ...' button. A warning message is displayed: 'Kompromittiertes, leicht zu erratendes Passwort'. Below the warning is a button 'Passwort auf Website ändern'. At the bottom, there are fields for 'Benutzername' (containing '@byom.de') and 'Passwort' (masked with dots), and a 'Website' field containing 'www.amazon.de'.

amazon.de
Zuletzt geändert 04.12.23

Bearbeiten ...

Kompromittiertes, leicht zu erratendes Passwort

Dieses Passwort wurde bei einem Datenleck gefunden. Dadurch besteht große Gefahr, dass dieser Account kompromittiert wird.

Passwort auf Website ändern

Benutzername @byom.de

Passwort

Website www.amazon.de

Speicherung von Passwörtern: verbessertes Hashing

- **Salt:** zufälliger Wert pro User optional: **Dienst-Kennung** (fester Wert)
- **Rundenzahl N :** kann bei Bedarf erhöht werden

- *MD5crypt* gilt mittlerweile als unsicher:

[\[http://phk.freebsd.dk/sagas/md5crypt_eol.html\]](http://phk.freebsd.dk/sagas/md5crypt_eol.html)

- 8 Zeichen Salt (à 6 Bit)
- Rundenzahl: $N = 1002$ fest, MD5
- 128 Bit Output

```
openssl passwd -1 Test
```

```
$1$nhjHwZ8y$fvG3gIpiGpk7S07AlBche0
```

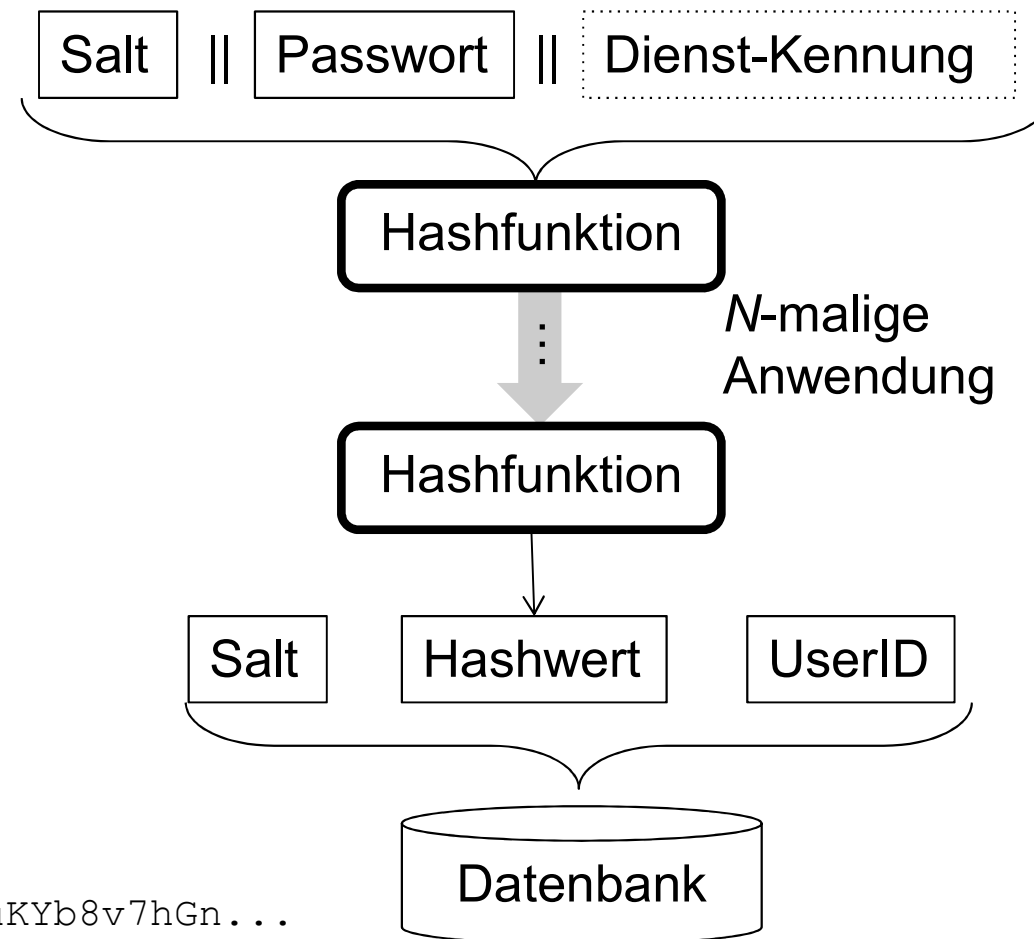
- besser: *SHA-crypt* (SHA-256, SHA-512):

[\[http://www.akkadia.org/drepper/SHA-crypt.txt\]](http://www.akkadia.org/drepper/SHA-crypt.txt)

- Salt: 16 Zeichen
- Rundenzahl: $1000 \leq N < 10^9$ variabel
- 256 (512) Bit Output

```
$5$rounds=5000$saltsaltsaltsalt$5RbfsOruKYb8v7hGn...
```

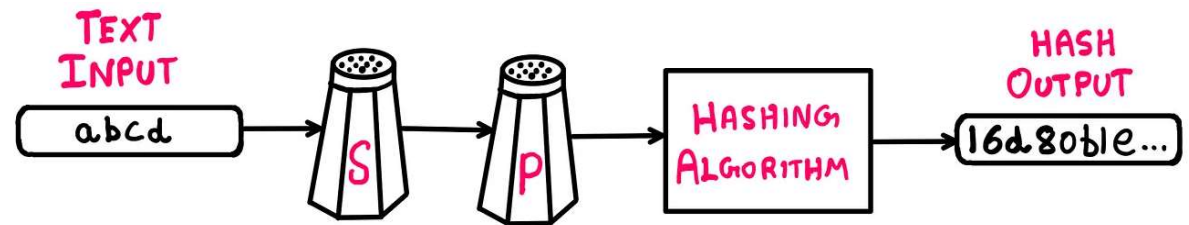
- moderne Verfahren: siehe unten



Verwendung von “Pepper”

- Salt schützt gegen Angriffe mit Rainbow-Tables
- wenn Angreifer aber Kopie der Datenbank erhält (z.B. per SQL-Injection), dann ist Brute-Force-Angriff auf (schwache) Passwörter dennoch möglich

- Schutz dagegen: “Pepper” = systemweites Secret



- Realisierung z.B.
 - bei erster Anwendung der Hashfunktion: $h(\text{salt} + \text{passwd} + \text{pepper})$
 - nachgelagert: $h(\text{pepper} + \text{hash})$ oder $\text{MAC}_{\text{pepper}}(\text{hash})$
- Anforderung: Pepper muss geheim gehalten werden (z.B. in HSM, jedenfalls getrennt von DB)
- beachte: Wechsel des Pepper nicht möglich, ohne dass sich *alle* User einmal einloggen

Abbildung: <https://blog.compliant.io/added-security-with-salt-and-pepper-a-recipe-for-securer-storage-4a884a060b9b>

Erhöhung der Komplexität zum Schutz gegen Brute-Force

- **Spezialhardware** beschleunigt *Brute-Force* Passwort-Cracking erheblich (diese wird auch typischerweise für Crypto-Mining verwendet)

- GPU (Graphics Processing Unit) – unterstützt von ->
- FPGA (field-programmable gate array)
- ASIC (application-specific integrated circuit)



[\[https://hashcat.net\]](https://hashcat.net)

- 8-Zeichen-Passwort Windows NTLM mit 8 GPUs in < 2,5h zu knacken (2/2019)

[\[https://www.theregister.co.uk/2019/02/14/password_length/\]](https://www.theregister.co.uk/2019/02/14/password_length/)

- **Durchsatz** hängt stark vom Hash-Algorithmus ab,
[\[http://passwords12.at.ifi.uio.no/Jeremi_Gosney_Password_Cracking_HPC_Passwords12.pdf\]](http://passwords12.at.ifi.uio.no/Jeremi_Gosney_Password_Cracking_HPC_Passwords12.pdf)

NTLM	$3,48 * 10^{11}/s$	Faktor 10^6
sha512crypt	$3,64 * 10^5/s$	

- **Ziel:** Verfahren mit nachweislich *hohem Speicherbedarf*
(Motivation: Arbeitsspeicher ist teurer zu vergrößern als #Operationen/s)

„*memory-hard functions*, which require a large amount of memory to be computed, and impose computational penalties if less memory is used.“

- **aktuelle Empfehlungen:**

- Scrypt [\[https://tools.ietf.org/html/draft-josefsson-scrypt-kdf-01\]](https://tools.ietf.org/html/draft-josefsson-scrypt-kdf-01)
- Argon2, Sieger Password Hashing Competition, [\[https://password-hashing.net\]](https://password-hashing.net)

2/2023: Review NIST SP 800-132 Recommendation for Password-Based Key Derivation
[\[https://csrc.nist.gov/News/2022/public-comments-requested-on-sp-800-132\]](https://csrc.nist.gov/News/2022/public-comments-requested-on-sp-800-132)

Einfluss des Verfahren auf Durchsatz bei Brute-Force

Estimated Password Recovery Times — 1x Terahash Brutalis, 44x Terahash Inmanis (448x Nvidia RTX 2080)

Full US keyboard mask attack with Terahash Hashstack

NTLM	31.82 TH/s	Instant	Instant	Instant	Instant	3 mins 29 secs	5 hrs 30 mins	3 wks 0 day	5 yrs 7 mos	538 yrs 1 mo	51.2 mil
MD5	17.77 TH/s	Instant	Instant	Instant	Instant	6 mins 14 secs	9 hrs 50 mins	1 mo 1 wk	10 yrs 1 mo	963 yrs 4 mos	91.6 mil
NetNTLMv1 / NetNTLMv1+ESS	16.82 TH/s	Instant	Instant	Instant	Instant	6 mins 35 secs	10 hrs 24 mins	1 mo 1 wk	10 yrs 8 mos	1 mil	96.8 mil
LM	15.81 TH/s	Instant	Instant	Instant	Instant						
SHA1	5.89 TH/s	Instant	Instant	Instant	Instant	18 mins 47 secs	1 day 5 hrs	3 mos 3 wks	30 yrs 7 mos	2.9 mil	276.3 mil
SHA2-256	2.42 TH/s	Instant	Instant	Instant	Instant	45 mins 39 secs	3 days 0 hr	9 mos 1 wk	74 yrs 4 mos	7.1 mil	671.9 mil
NetNTLMv2	1.22 TH/s	Instant	Instant	Instant	Instant	1 hr 30 mins	5 days 23 hrs	1 yr 6 mos	147 yrs 10 mos	14.1 mil	1335.5 mil
SHA2-512	801.9 GH/s	Instant	Instant	Instant	1 min 28 secs	2 hrs 17 mins	1 wk 2 days	2 yrs 4 mos	224 yrs 9 mos	21.4 mil	2029.7 mil
descript, DES (Unix), Traditional DES	647.59 GH/s	Instant	Instant	Instant	1 min 48 secs	2 hrs 50 mins	1 wk 4 days	2 yrs 11 mos	278 yrs 3 mos	26.5 mil	2513.3 mil
Kerberos 5, etype 23, TGS-REP	206.97 GH/s	Instant	Instant	Instant	5 mins 38 secs	8 hrs 54 mins	1 mo 0 wk	9 yrs 2 mos	870 yrs 10 mos	82.8 mil	7864 mil
Kerberos 5, etype 23, AS-REQ Pre-Auth	206.78 GH/s	Instant	Instant	Instant	5 mins 38 secs	8 hrs 54 mins	1 mo 0 wk	9 yrs 2 mos	871 yrs 8 mos	82.9 mil	7871.2 mil
md5crypt, MD5 (Unix), Cisco-IOS \$1\$ (MD5)	7.61 GH/s	Instant	Instant	1 min 37 secs	2 hrs 33 mins	1 wk 3 days	2 yrs 7 mos	249 yrs 5 mos	23.7 mil	2252.6 mil	213995.1 mil
LastPass + LastPass sniffed	1.78 GH/s	Instant	Instant	6 mins 52 secs	10 hrs 52 mins	1 mo 1 wk	11 yrs 2 mos	1.1 mil	101.1 mil	9600.8 mil	912079.6 mil
macOS v10.8+ (PBKDF2-SHA512)	335.09 MH/s	Instant	Instant	36 mins 34 secs	2 days 9 hrs	7 mos 2 wks	59 yrs 7 mos	5.7 mil	538.2 mil	51127.7 mil	4857134 mil
WPA-EAPOL-PBKDF2	277.23 MH/s					9 mos 0 wk	72 yrs 0 mo	6.8 mil	650.5 mil	61799.3 mil	5870931.8 mil
TrueCrypt RIPEMD160 + XTS 512 bit	211.78 MH/s	Instant	Instant	57 mins 52 secs	3 days 19 hrs	11 mos 3 wks	94 yrs 3 mos	9 mil	851.6 mil	80899.5 mil	7685455.6 mil
7-Zip	181.51 MH/s	Instant	Instant	1 hr 7 mins	4 days 10 hrs	1 yr 1 mo	110 yrs 0 mo	10.5 mil	993.6 mil	94389.2 mil	8966975.1 mil
sha512crypt \$6\$, SHA512 (Unix)	119.46 MH/s	Instant	1 min 5 secs	1 hr 42 mins	6 days 18 hrs	1 yr 9 mos	167 yrs 2 mos	15.9 mil	1509.7 mil	143419.6 mil	13624861.4 mil
DPAPI masterkey file v1	47.23 MH/s	Instant	2 mins 44 secs	4 hrs 19 mins	2 wks 3 days	4 yrs 5 mos	422 yrs 10 mos	40.2 mil	3818.1 mil	362723.1 mil	34458696.1 mil
RAR5	28.15 MH/s	Instant	4 mins 35 secs	7 hrs 15 mins	4 wks 0 day	7 yrs 5 mos	709 yrs 7 mos	67.4 mil	6407.6 mil	608720.6 mil	57828453.9 mil
DPAPI masterkey file v2	27.82 MH/s	Instant	4 mins 39 secs	7 hrs 20 mins	4 wks 1 day	7 yrs 6 mos	717 yrs 10 mos	68.2 mil	6482.1 mil	615797.6 mil	58500769.5 mil
RAR3-hp	20.84 MH/s	Instant	6 mins 12 secs	9 hrs 47 mins	1 mo 1 wk	10 yrs 1 mo	958 yrs 2 mos	91.1 mil	8652.3 mil	821972.3 mil	78087367.8 mil
KeePass 1 (AES/Twofish) and KeePass 2 (AES)	17.8 MH/s	Instant	7 mins 15 secs	11 hrs 28 mins	1 mo 2 wks	11 yrs 9 mos	1.1 mil	106.7 mil	10131.9 mil	962529.5 mil	91440305.8 mil
bcrypt \$2*\$, Blowfish (Unix)	11.37 MH/s	Instant	11 mins 21 secs	17 hrs 57 mins	2 mos 1 wk	18 yrs 5 mos	1.8 mil	167 mil	15860.3 mil	1506727.9 mil	143139150.9 mil
Bitcoin/Litecoin wallet.dat	3.55 MH/s	Instant	36 mins 18 secs	2 days 9 hrs	7 mos 2 wks	59 yrs 1 mo	5.6 mil	534.1 mil	50743.7 mil	4820655.6 mil	457962282.7 mil
	Speed	Length 4	Length 5	Length 6	Length 7	Length 8	Length 9	Length 10	Length 11	Length 12	Length 13

[Quelle: <https://billatnapier.medium.com/the-proper-password-hasher-and-memory-buster-argon2-3d5924a9d5b4>],

Original [TERAHASH, vor 2022]

UNCOMMON (NON-GIBBERISH) BASE WORD ORDER UNKNOWN

Tr0ub4dor &3

CAPS? COMMON SUBSTITUTIONS NUMERAL PUNCTUATION

(YOU CAN ADD A FEW MORE BITS TO ACCOUNT FOR THE FACT THAT THIS IS ONLY ONE OF A FEW COMMON FORMATS.)

~28 BITS OF ENTROPY

$2^{28} = 3 \text{ DAYS AT } 1000 \text{ GUESSES/SEC}$

(PLAUSIBLE ATTACK ON A WEAK REMOTE WEB SERVICE. YES, CRACKING A STOLEN HASH IS FASTER, BUT IT'S NOT WHAT THE AVERAGE USER SHOULD WORRY ABOUT.)

DIFFICULTY TO GUESS: **EASY**

WAS IT TROMBONE? NO, TROUBADOR. AND ONE OF THE 0s WAS A ZERO?

AND THERE WAS SOME SYMBOL...

DIFFICULTY TO REMEMBER: **HARD**

correct horse battery staple

FOUR RANDOM COMMON WORDS

~44 BITS OF ENTROPY

$2^{44} = 550 \text{ YEARS AT } 1000 \text{ GUESSES/SEC}$

DIFFICULTY TO GUESS: **HARD**

THAT'S A BATTERY STAPLE.

CORRECT!

DIFFICULTY TO REMEMBER: YOU'VE ALREADY MEMORIZED IT

THROUGH 20 YEARS OF EFFORT, WE'VE SUCCESSFULLY TRAINED EVERYONE TO USE PASSWORDS THAT ARE HARD FOR HUMANS TO REMEMBER, BUT EASY FOR COMPUTERS TO GUESS.

entsprechender
Passwort-
Generator
(ohne Gewähr):

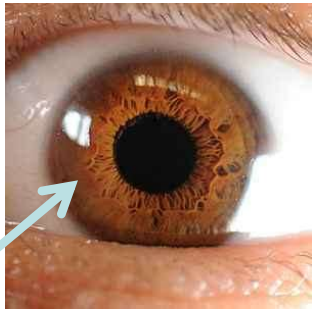
Biometrie

- Biometrie = Lehre von der Vermessung von Eigenschaften des Körpers.
- Charakteristische Merkmale werden oftmals zur **kombinierten Identifizierung und Authentifizierung** verwendet.
- Merkmale werden dabei durch Messung bestimmt und mit hinterlegtem **Referenzwert** verglichen.

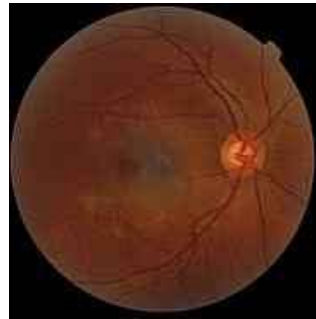
Problem: im Gegensatz zu Passwort-Überprüfung kann nicht auf 100%ige Übereinstimmung getestet werden.

Beispiele:

Iris (Regenbogenhaut)



Retina (Netzhaut)



Fingerabdruck



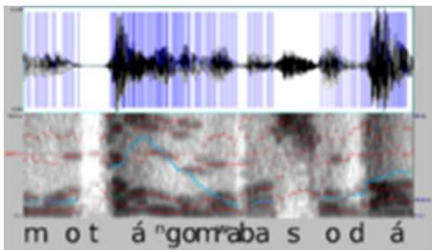
Gesicht



Handvenen



Stimme



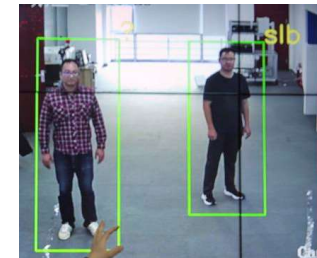
Unterschrift

A handwritten signature in black ink, which appears to read 'B. Franklen'.

Tippverhalten



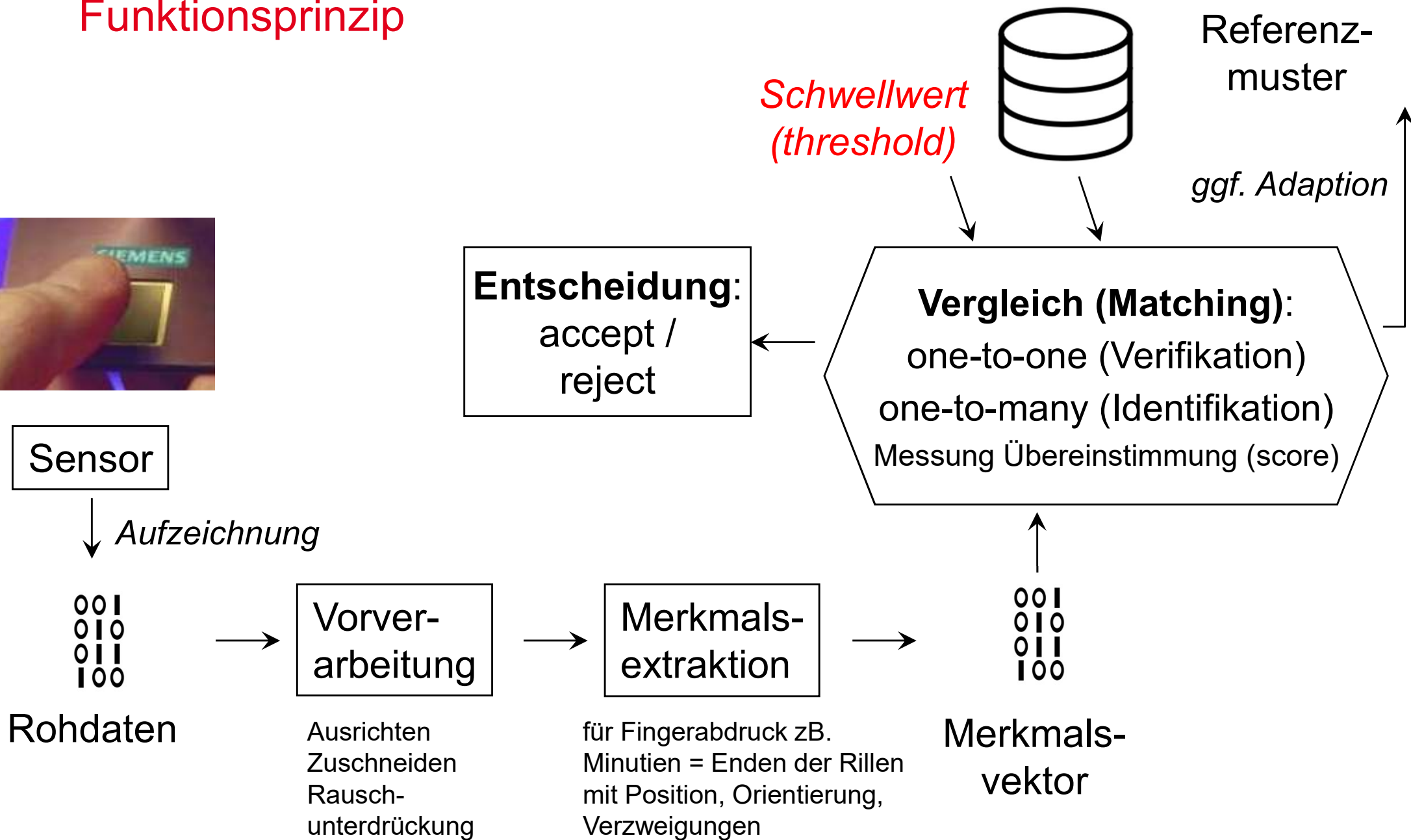
Gangerkennung



Art der Verfahren und Anforderungen

- Unterscheidung:
 - physiologische, **statische** Verfahren (z.B. Fingerabdruck)
 - verhaltenstypische, **dynamische** Verfahren (z.B. Stimme)
- Merkmal ...
 - muss bei möglichst vielen Menschen vorkommen
-> **Universalität**
 - darf bei verschiedenen Menschen nicht gleich ausgeprägt sein
-> **Eindeutigkeit**
 - verändert sich über gewissen Zeitraum in seiner Ausprägung nicht
-> **Konstanz**
 - lässt sich technisch über Sensoren erfassen, auf kostengünstige Weise und mit hinreichend niedriger Fehlerrate -> **Messbarkeit**
 - ist hinreichend komfortabel, vertraut und akzeptiert
-> **Anwenderfreundlichkeit**
 - kann nur schwer imitiert oder gestohlen werden
-> **Fälschungssicherheit**

Funktionsprinzip

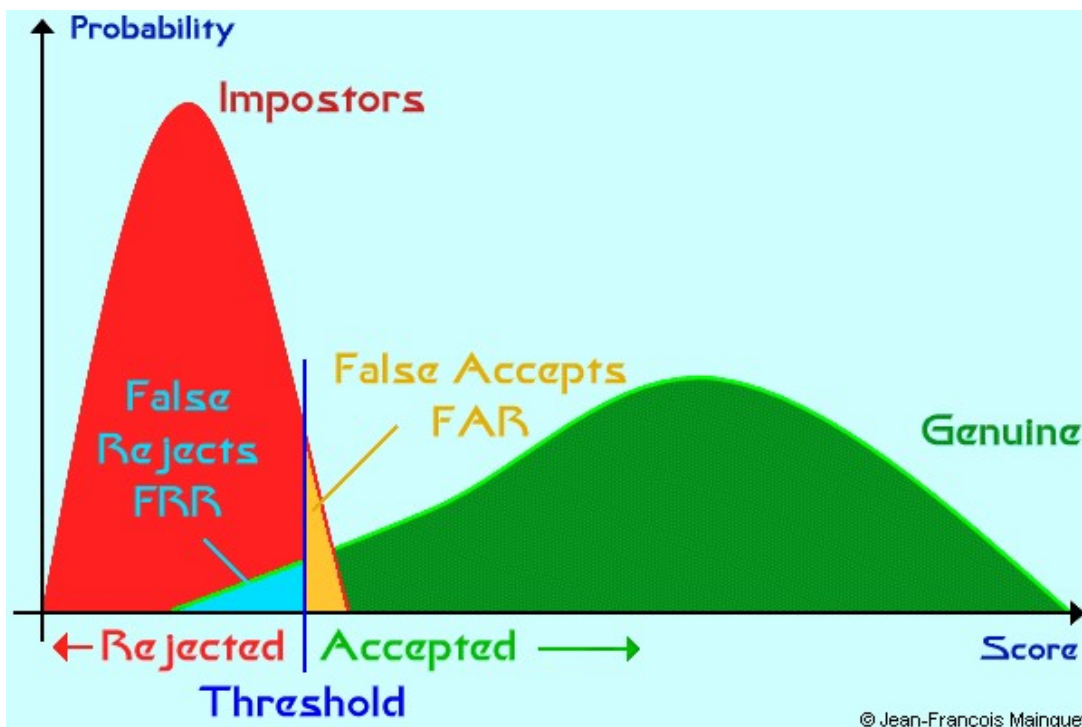
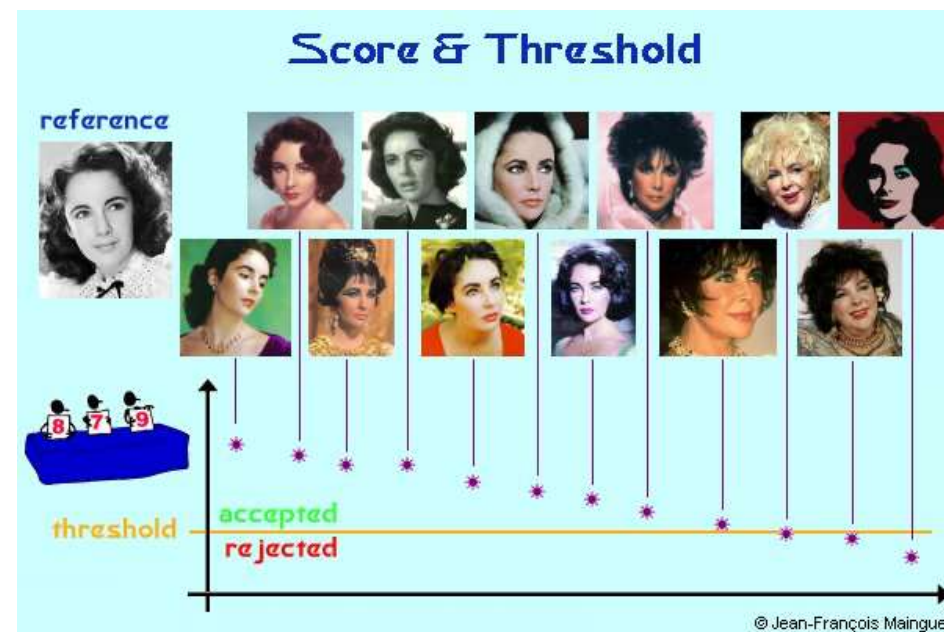


Fehlertypen

Gütemaß:

- False Acceptance Rate (FAR) – wie häufig wird Fremder akzeptiert
- False Rejection Rate (FRR) – wie häufig wird legitimer User abgelehnt

[Abb.: <https://biometrics.mainguet.org/basics/accuracy.htm>]



FAR & FRR

	rejected 	accepted 
100 genuines 	1 rejected FRR=1% 	99 accepted TAR=99% 
1000 impostors 	999 rejected TRR=99.9% 	1 accepted FAR=0.1% 

© Jean-François Mainguet

Güte verschiedener biometrischer Verfahren

Quelle: <https://www.itwissen.info/Fingerabdruckerkennung-fingerprint-identification.html> (2019)

Biometrisches Verfahren	FAR-Werte in %	FRR-Werte in %
Fingerabdruck	0,001 ... 2	0,1 ... 5
Iriserkennung	0,0001 ... 1	0,1 ... 2
Gesichtserkennung	0,5 ... 2	1 ... 3
Handgeometrie	1 ... 5	1 ... 5

Quelle: [Conti, Militello, Vitabile, 2017]

Table 2 Comparison between biometric technologies

Biometric Technology	Frr range (%)	Far range (%)	Cost	Template dimension (Bytes)	User acceptance
Face geometry	10÷20	0.001÷1	Medium	Few bytes	High
Hand geometry	1÷10	1	Medium	<10	Medium
Iris scansion	1÷10	~0	High	512	Low
Retina scansion	1	0.01	Very High		Low
Voice	10÷20	2÷5	Low	1500	High
Calligraphy	3÷10	1	Medium	1500	High
Fingerprint	3÷7	0.0001÷0.001	Medium	300÷1200	Medium

Angriffsmöglichkeiten



Foto mit Fingerabdruck

[[Starbug, 2014](#)]

Iris-Erkennung

[[Starbug, 2014](#)]



Biometrie

Hacker tricksen Venenscanner mit Wachs aus

Der BND benutzt sie, in asiatischen Geldautomaten stecken sie auch: Venenerkennungssysteme. Der Biometrie-Hacker Starbug hat einen Weg gefunden, sie zu überlisten - mit filmreifen Methoden.

[Starbug/Albrecht, 2018]

Von **Patrick Beuth**

27.12.2018, 19.17 Uhr

Biometrie

Polizei lässt Fingerabdruck eines Toten nachbauen

Die US-Polizei will das Smartphone eines Mordopfers entsperren, um darin nach Hinweisen auf den Täter zu suchen. Dafür wollen sie den Fingerabdrucksensor austricksen.

Von **Patrick Beuth**

[[Die Zeit, 2016](#)]

22. Juli 2016, 17:37 Uhr / [24 Kommentare](#) / [🔖](#)

CCC erbeutet Biometrie-Datenbank des US-Militärs

2022-12-27 06:20:48, linus

[[CCC, 2022](#)]

Das US-Militär hat massenhaft Geräte zur biometrischen Erfassung von Menschen in Afghanistan genutzt. Einige Geräte wurden beim hastigen Abzug der NATO-Truppen zurückgelassen. Forscher des CCC haben bei Analysen solcher Geräte große Mengen an biometrischen und weiteren personenbezogenen Daten

Angriff auf Windows Hello mit externer Kamera [[2021](#)]

Compromising Passwordless Solutions



1 The attacker captures or creates an Infra-Red (IR) image of the victim



2 The attacker creates a custom USB device that impersonates a USB camera device



3 The attacker saves the captured data on a custom USB device



4 The attacker connects the custom USB device to John's computer. The USB device transmits the captured IR image.



5 The biometrical system accepts the biometrical credentials and logs the user in



6 The attacker gets access to sensitive information and files of the victim

CYBERARK LABS

Risiko der Offenlegung von Merkmalen auf Social Media [[Trendmicro 2022](#)]

Leaked Today, Exploited for Life

How Social Media Biometric Patterns Affect Your Future

The photos, videos, and audio posts we put online expose sensitive biometric patterns that can be abused by cybercriminals. These patterns are virtually unchangeable and can be used now or in future attacks.

One-Time Password (OTP)-Verfahren

- **Motivation:** Einmalpasswörter werden genutzt, ...
 - wo eine Passwort-Eingabe riskant ist (Phishing, Abhören)
 - als weiterer Faktor bei 2FA (bei Erzeugung auf separatem Gerät)
- **Beobachtungen:** OTP-Verfahren
 - sind nicht anfällig gegen Mithören (passiver Angriff) und Wiedereinspielen (Replay-Angriff)
 - können aber durch **Man-in-the-Middle** angegriffen werden (sofern Server nicht authentifiziert wird)
 - limitieren den Angreifer (einmalige Gültigkeit, beschränktes Zeitfenster)
- **Zustand** zwischen User und Server muss übereinstimmen bei Zähler- oder Zeit-basierten Verfahren
 - > Mechanismus muss mit abgebrochenen Authentifikations-Vorgängen umgehen können und sollte Nach-Synchronisation erlauben
 - Challenge-Response-Verfahren vermeiden das Problem

Bsp: RSA SecurID



- synchrone Uhren zwischen Gerät und Server
- Gerät erzeugt alle 30 oder 60 Sekunden neuen Tokencode
- in Berechnung des Codes *per AES* gehen ein:
 - Zeit
 - Geräte-spezifisches Geheimnis (Seed, 128 Bit)
 - ggf. PIN

(Geräte haben Seriennummer, um Zuordnung zu Benutzer zu ermöglichen)

- verwendbar für **Zwei-Faktor-Authentifikation**, d.h. zusätzlich zu Benutzernamen und Passwort

- Hacker-Angriff 2011: [<http://heise.de/-1256298>]
 - Seeds und Seriennummern bei RSA Security erbeutet, vermutlich für Angriff auf Lockheed Martin genutzt
 - Austausch von 40 Mio. Tokens erforderlich!



[Grafik: <http://www.tokenguard.com/RSA-SecurID-SID900.asp>]

[Grafik: <http://de.wikipedia.org/wiki/SecurID>]

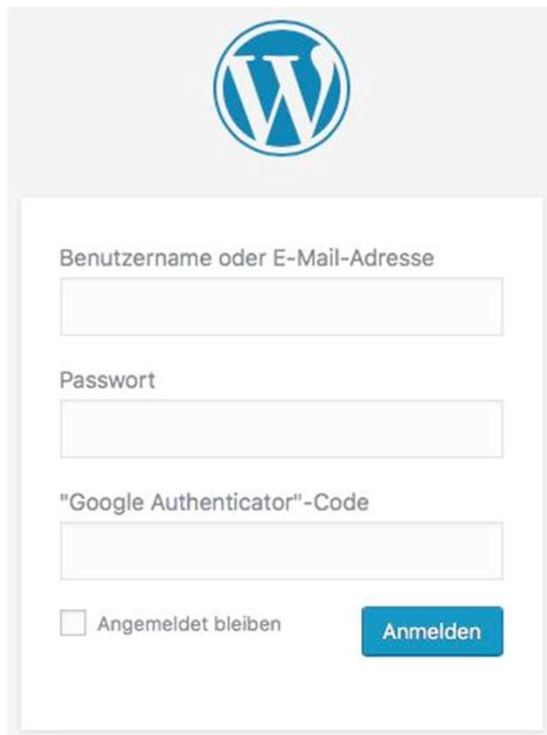
[Grafik: http://docstore.mik.ua/univercd/cc/td/doc/product/vpn/client/re14_0/osxguide/connect.htm]

Bsp: Google Authenticator

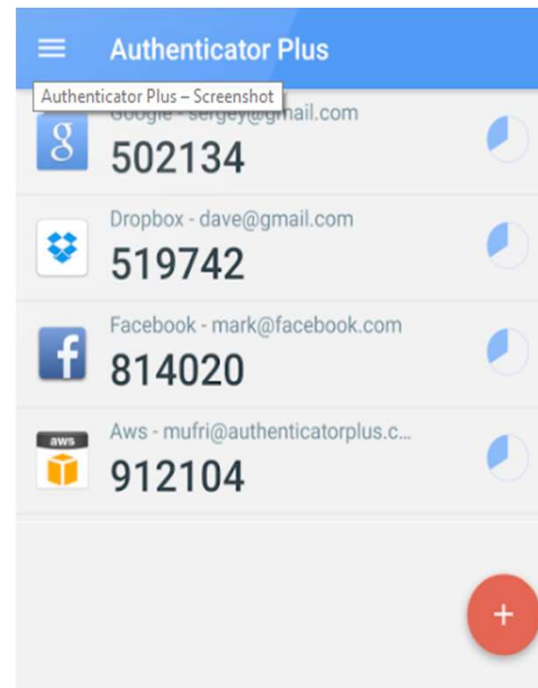
- OTP zusätzlich zu Benutzernamen + Passwort, damit 2-Faktor-Auth.
- basiert auf TOTP (siehe nächste Folie)

- z.B. WordPress-Login

App erzeugt periodisch OTP-Codes



The image shows the WordPress login interface. At the top is the WordPress logo. Below it are three input fields: 'Benutzername oder E-Mail-Adresse', 'Passwort', and '"Google Authenticator"-Code'. At the bottom left is a checkbox labeled 'Angemeldet bleiben' and a blue 'Anmelden' button.



Open Source-
Tool:



<http://freeotp.github.io>

[Grafiken: <https://technikram.net/2016/09/wordpress-zwei-faktor-authentifizierung-mit-google-authenticator-einrichten>,
<https://constey.de/2017/05/google-authenticator-token-exportierenimportieren/>]

HOTP / TOTP: Zähler- bzw. Zeit-basiert

- basierend auf **Counter C**:
 - HMAC-based [\[RFC 4226\]](#): **HOTP(K,C) = Truncate(HMAC-SHA-1_K(C))**
 - HMAC ist Standard-Konstruktion, um aus Hashfunktion h sicher einen MAC zu erhalten [\[RFC 2104\]](#)
$$\text{HMAC}_K(x) := h(K \oplus \text{opad} \parallel h(K \oplus \text{ipad} \parallel x))$$

h: Hashfunktion

K: Key (hier: Shared Secret zwischen User und Server),
opad, ipad: konstante Strings zum Auffüllen (in HMAC standardisiert)
 - Truncate: Abbildung auf Dezimalzahl gewünschter Länge (6 – 8 Ziffern)
- basierend auf **Zeit**
 - Time-Based [\[RFC 6238\]](#): **TOTP = HOTP(K, T)**, wo
 - $T = \lfloor \frac{T_{\text{current}} - T_0}{X} \rfloor$ Zeit in Sekunden,
 - $T_{\text{current}} - T_0$: Unix Epoch Time (= Sekunden seit 1.1.1970),
 - X: Schrittweite (Default: 30)

Kryptographisches Challenge-Response-Verfahren

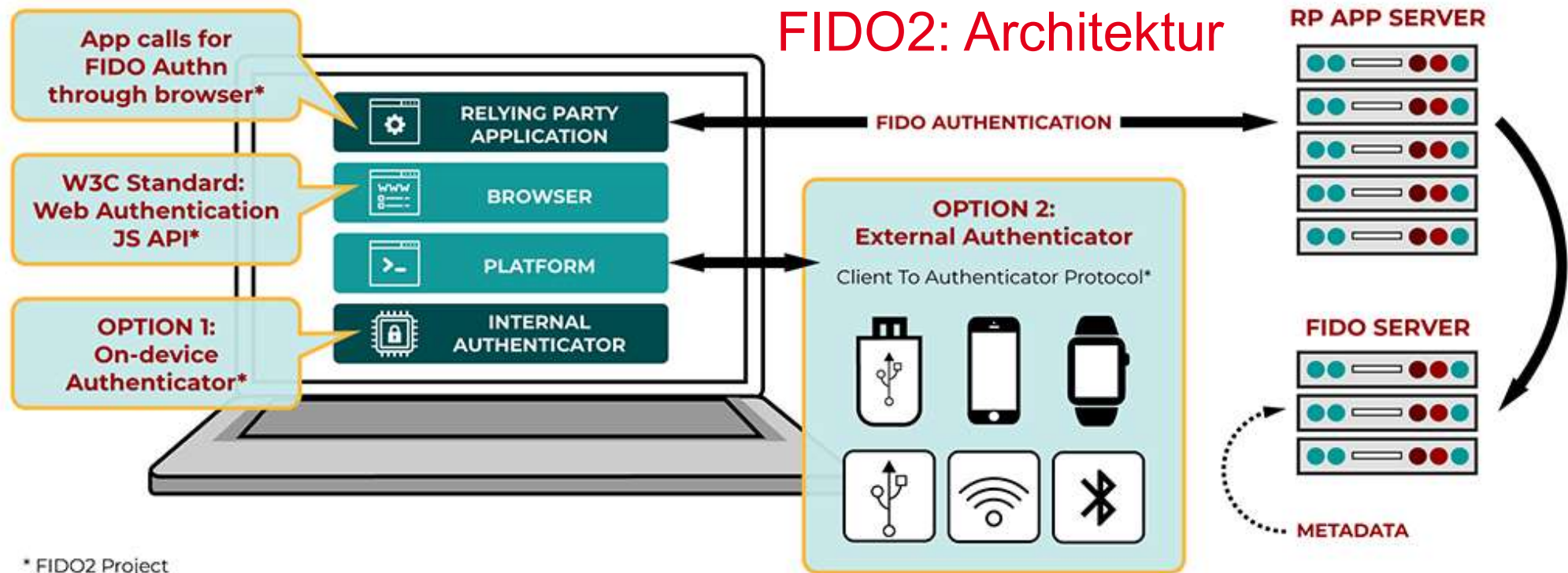
- **symmetrische Verfahren:**
 - Client und Server haben gemeinsames Geheimnis
 - Nachweis der Kenntnis durch Verschlüsseln
- **asymmetrische Verfahren** (Variante: Zero-Knowledge -> relativ aufwändig)
 - Server speichert Public Key des Clients
 - Nachweis der Kenntnis des Private Key durch Signieren
- Sicherheit:
 - Herausforderung *Schlüsselverwaltung*:
 - sym.: vertrauliche Speicherung
 - asym.: Schutz gg. Veränderung (Alternative: Client weist Zertifikat vor)
 - Angriffe bei sym. Verfahren: *Known-Plaintext*, *Wörterbuch-Angriff*
 - *Replay-Attack* möglich, falls sich Zufallszahlen wiederholen
 - *MITM-Angriff* denkbar, falls keine beidseitige Authentifikation: Angreifer kann sich als Server ausgeben

Bsp: FIDO



- Reihe von offenen Standards
- **U2F (Universal 2nd Factor, CTAP1):** ergänzt Benutzername + Passwort,
 - Hardware-Token (i.d.R. via USB, kein Treiber erforderlich)
 - Private Keys sind nicht auslesbar
 - Anwesenheit des Users wird durch Knopfdruck bestätigt
- **UAF (Universal Authentication Factor):**
 - passwortlose (und ggf. benutzerdatenlose) Authentifizierung,
 - Absicherung des Tokens durch PIN oder Biometrie
- **aktuell: FIDO2:**
 - W3C-Standard [WebAuthn](https://www.w3.org/TR/webauthn/) Level 2: JS-API für Credential-Erstellung/-Nutzung
 - a) in Software realisierbar (weniger sicher) oder
 - b) als *“platform authenticator”*: im Gerät verbaut (TPM, Secure Element)
 - c) als *„roaming authenticator”*: auf einem anderen Gerät, Kommunikation per USB, Bluetooth Low Energy, NFC
 - Client to Authenticator Protocol (CTAP2) über USB-HID, BLE, NFC

[Bildquellen: <https://fidoalliance.org>,
https://commons.wikimedia.org/wiki/File:KEY-ID_FIDO.gif]



RP = Relying Party: Webserver, auf dem Anmeldung erfolgt

„FIDO Server:“ Software Library für WebAuthn-Serverkomponente beim RP

Metadata Service:

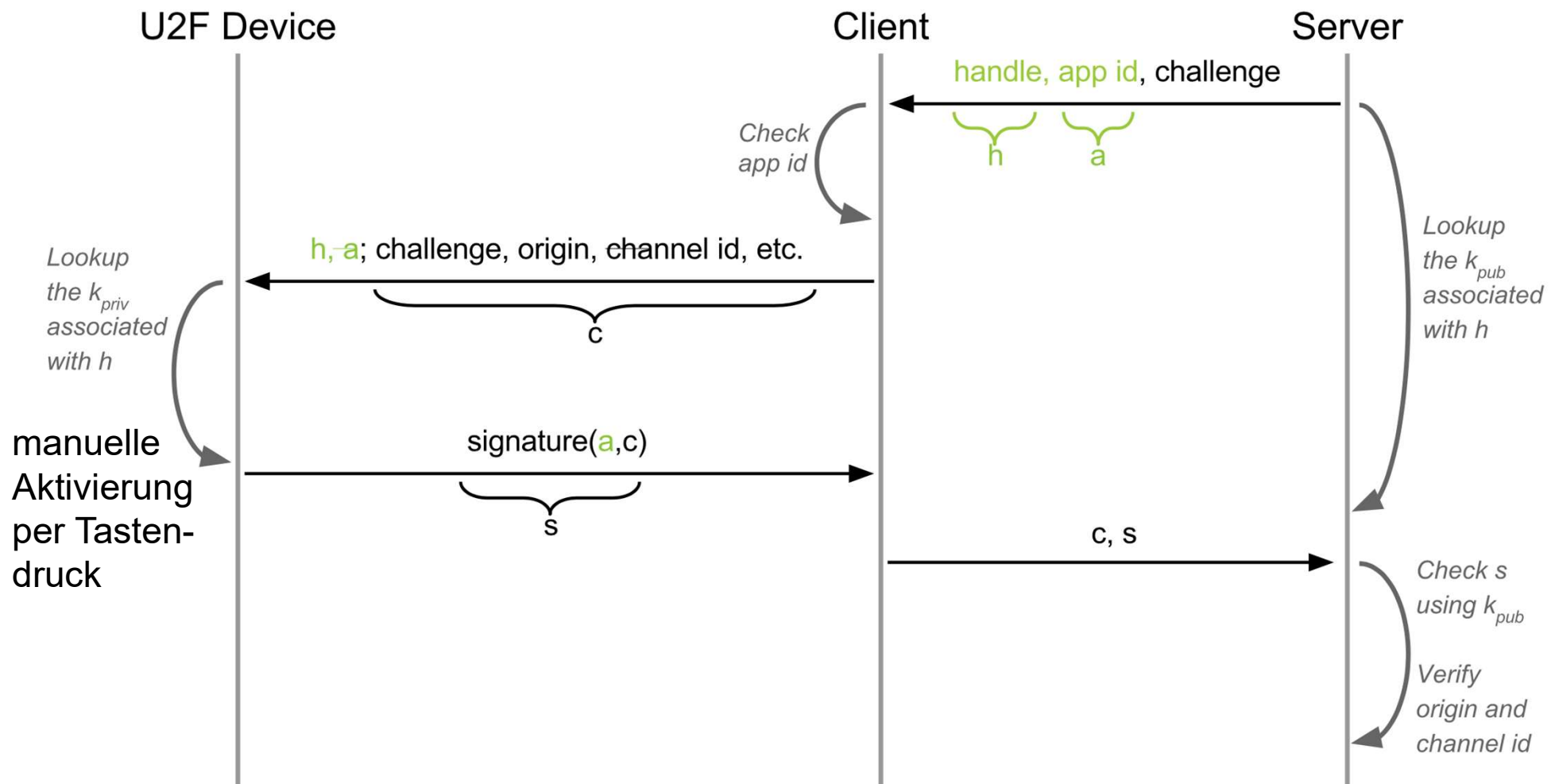
- stellt Informationen über alle zertifizierten Tokens bereit (z.B. Zertifikat des Herstellers, unterstützte Algorithmen)
- damit Echtheitsnachweis der Tokens
- Details: [<https://fidoalliance.org/metadata/>]

FIDO2: Sicherheitseigenschaften

- Kryptographische Verfahren:
 - *Challenge-Response-Protokoll* zwischen Relying Party (RP) und Device
 - kein Shared Secret notwendig (wie bei TOTP)
 - nutzt RSA- oder ECDSA-Signaturen -> *Brute-Force-Angriff aussichtslos*
- *Nutzer- und Datenschutz-freundlich:*
 - *Selbstregistrierung* möglich mit Erzeugung eines Schlüsselpaars
 - *Nicht-Verkettbarkeit:* pro Dienst eigenes Key Pair, auch mehrere möglich, Nutzer können sich somit Device teilen
 - Response wird digital übertragen, kein Abtippen nötig
- Schutz gegen *Phishing*: Schlüsselpaar und Response sind an Dienst gebunden, können nicht wiederverwendet werden
- Möglichkeit zum Nachweis des Device-Typs („attestation“)

Protokoll-Ablauf Authentifikation

Demo: [WebAuthn.me](https://webauthn.me), [WebAuthn.io](https://webauthn.io)



- *handle*: eindeutige Kennung des registrierten Public Key
- *app id*: z.B. <https://login.example.com> oder `android:apk-key-hash:585215fd51`

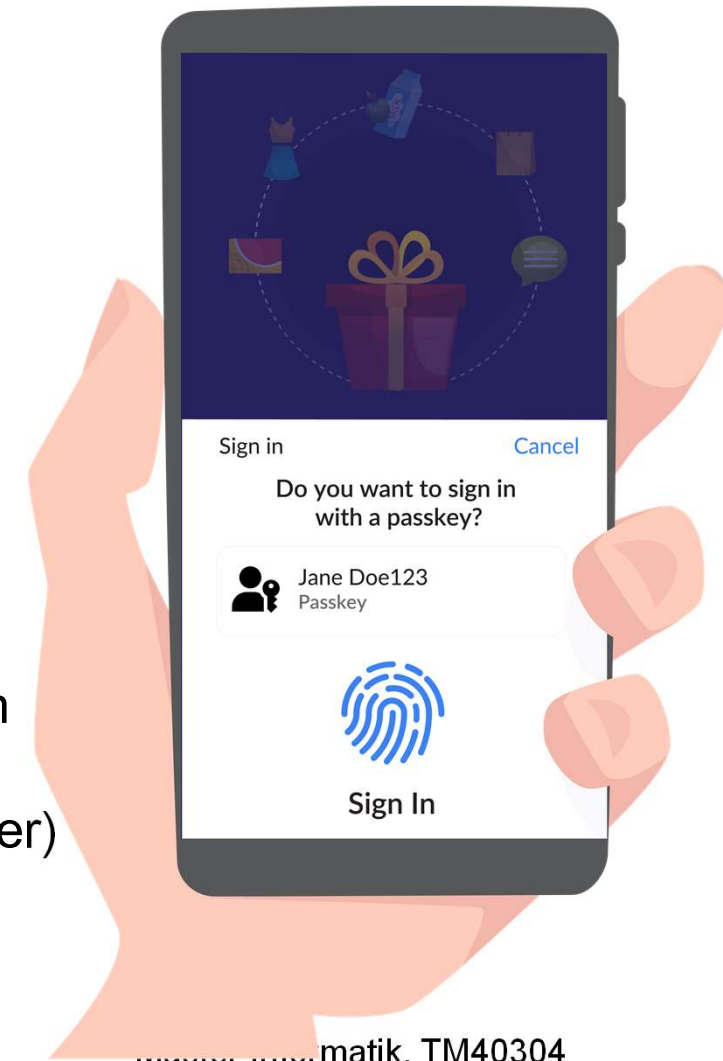
[Bildquelle: https://developers.yubico.com/U2F/Protocol_details/Overview.html]

Passkeys

„The beginning of the end of the password“

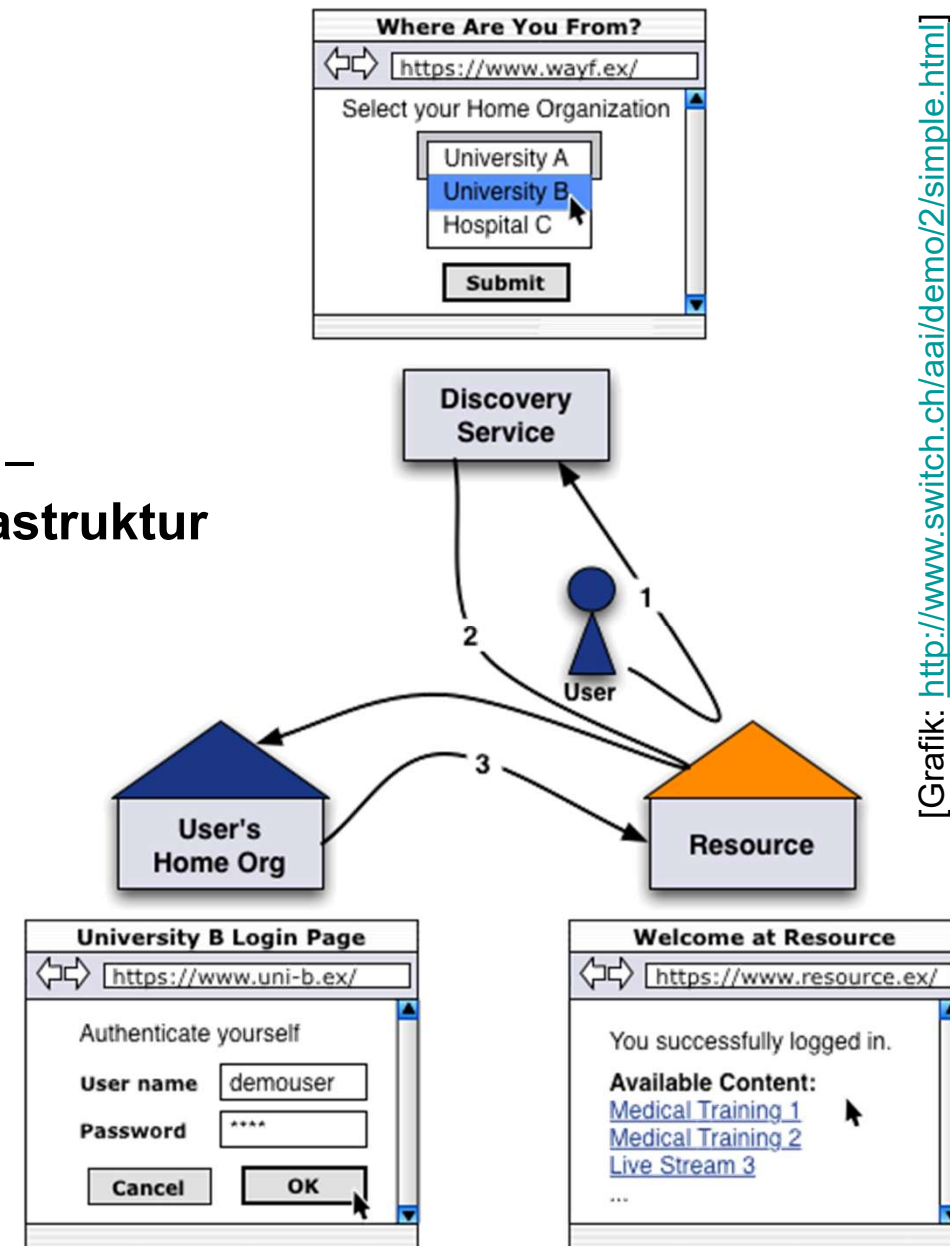
[[Google](#), 2023]

- Einschränkungen bei U2F:
 - physische Tokens erforderlich (verursachen Kosten)
 - Enrollment für jeden Dienst einzeln erforderlich
 - Umgang mit Token-Verlust
- WebAuthn “Level 3”:
 - *Smartphone* als „roaming authenticator“, welches mit PC per Bluetooth kommuniziert
-> stellt räumliche Nähe sicher, verhindert Phishing
 - „*multi-device FIDO credentials*“:
Übertragung der Credentials von altem auf neues Smartphone, Synchronisation durch Betriebssystem und Cloud
(Apple: iCloud Keychain, Google: Password Manager)
[<https://fidoalliance.org/white-paper-multi-device-fido-credentials/>]



Single-Sign On mit Shibboleth

- typischerweise von Hochschulen genutzt (Service Provider: Verlage)
- neben Authentifikation auch Autorisierung, z.B. Zugehörigkeit zu bestimmter Gruppe, sog. **AAI – Authentifizierungs- und Autorisierungs-Infrastruktur**
 - IdP: *Heimatinrichtung*, bei der Benutzer einen Account hat, bildet sog. **Föderation** mit
 - RP: Service-, Ressourcen-Anbieter
 - *Where-Are-You-From (WAYF)-Service*:
 - zentral gepflegte Liste der IdP, um Umleitung zu erleichtern (Service Provider kann aber explizit zugelassene IdP definieren)
 - z.B. von DFN betrieben, Beispiel siehe: <https://www.elab.moodle.elearning.lmu.de/auth/shibboleth/index.php>



[Grafik: <http://www.switch.ch/aai/demo/2/simple.html>]

SAML

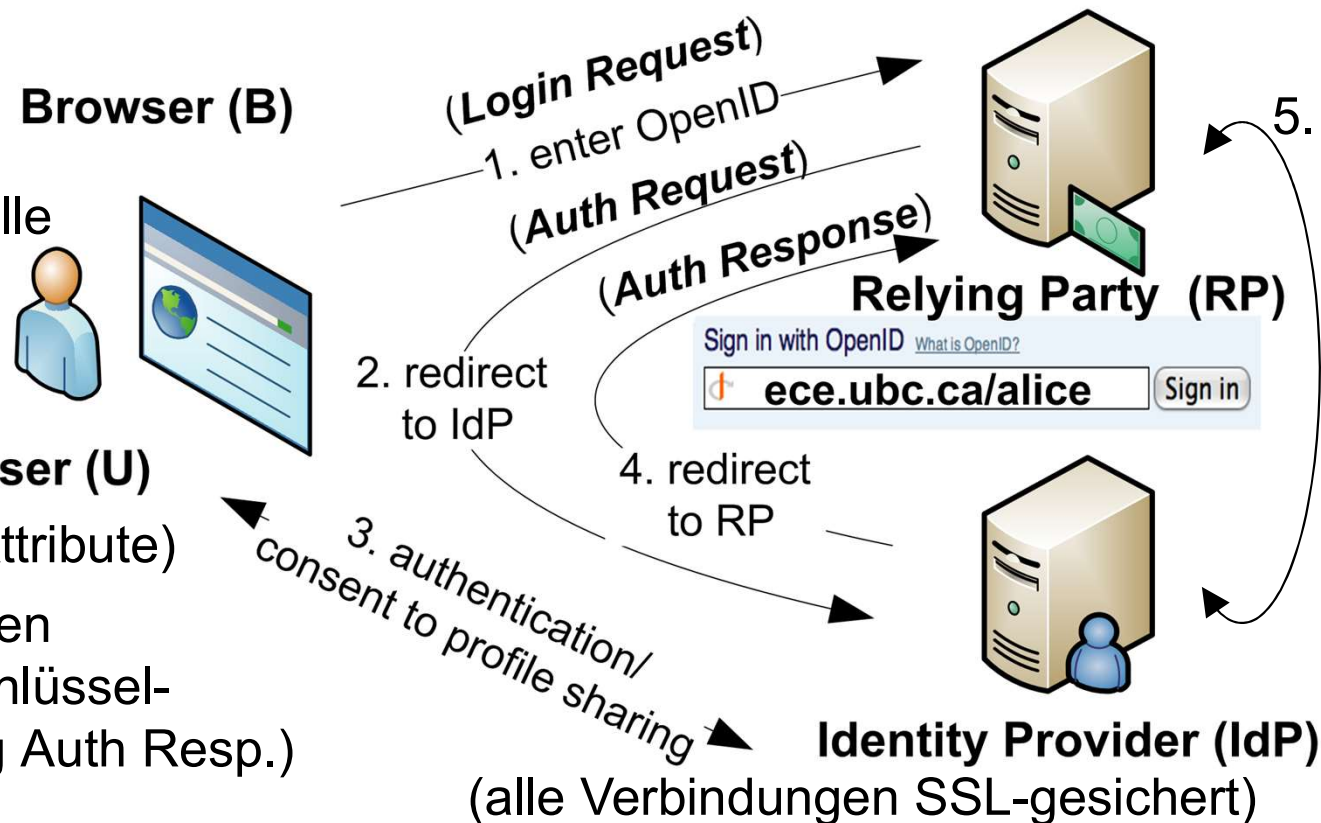
- Security Assertion and Markup Language, aktuelle Version: 2.0
[<https://www.oasis-open.org/standards#samlv2.0>] (Beispiele in [Eckert, Abschnitt 13.7.4] basieren auf SAML 1.1)
 - von Shibboleth genutzt in Schritt 3 zwischen Identity und Service Provider
 - Service Provider = „Relying Party“, vertraut Angaben des
 - Identity Provider = „Asserting Party“ (auch: SAML Authority)
 - XML-Datenstruktur mit optionaler Verschlüsselung und/oder Signatur gemäß W3C-Standards XML Encryption und XML Signature
[<http://www.w3.org/TR/xmlenc-core/>, <http://www.w3.org/TR/xmlsig-core/>]
 - bei Web-SSO über Browser des Benutzers ausgetauscht, daher Absicherung erforderlich
- Typen von Assertions:
 - Authentifikation („um wen handelt es sich?“, „wann und wie wurde er authentifiziert?“)
 - Autorisierung („darf Subjekt auf bestimmte Resource zugreifen?“)
 - Attribute (Zusatzangaben zum Subjekt, z.B. Kreditlimit)

Single-Sign On am Beispiel OpenID

- dezentralisiertes SSOVerfahren für das WWW: Benutzer ...
 - hat Account bei *Identity Provider* (IdP, bei OpenID auch: “OP”)
 - will damit Dienst eines *Service Providers* (Relying Party, RP) nutzen

- Ablauf:

1. GET/POST-Request an RP
2. 302-Redirect,
GET-Request an IdP
3. Übermittlung Passwort, Kontrolle
der zu übermittelnden
Parameter und des Ziels
4. 302-Redirect an RP,
GET-Request mit Angaben
zum Benutzer (freigegebene Attribute)
5. direkte Kommunikation zwischen
RP und IdP: Diffie-Hellman Schlüssel-
vereinbarung (zur Überprüfung Auth Resp.)



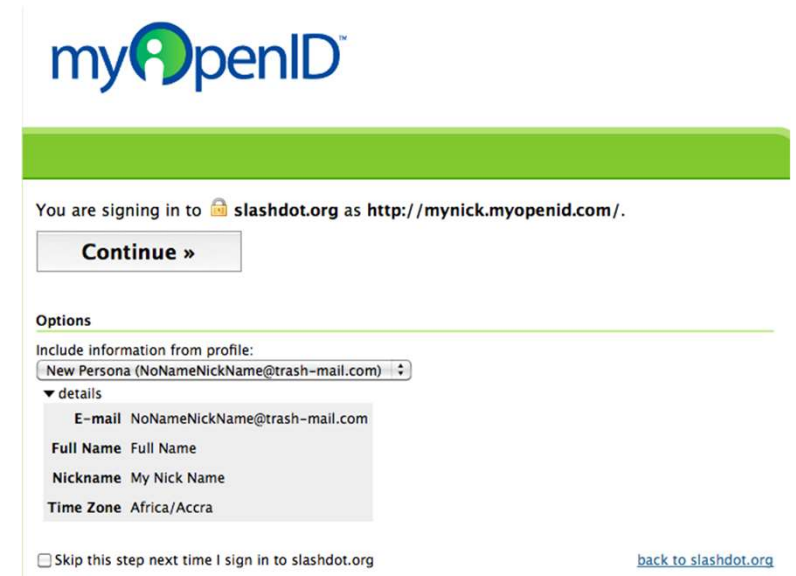
[Grafik: http://konstantin.beznosov.net/professional/wp-content/uploads/2012/09/how_openid_work.png]

OpenID – Bewertung

- Vorteile:
 - Nutzer kontrolliert übertragene Daten, kann mit versch. Teil-Identitäten agieren
 - IdP frei wählbar (sogar Betrieb eines eigenen IdP möglich)
 - IdP unabhängig von RP (zumindest in der Theorie)
 - OpenID-Provider kann starkes Authentifikations-Verfahren anbieten
 - relativ leicht implementierbar

- Nachteile:
 - IdP ist interessantes Phishing-Ziel
 - IdP sieht Surfverhalten des Benutzers
 - RP muss sich auf IdP verlassen oder User-Identität selbst prüfen (etwa bei Bezahlendiensten)

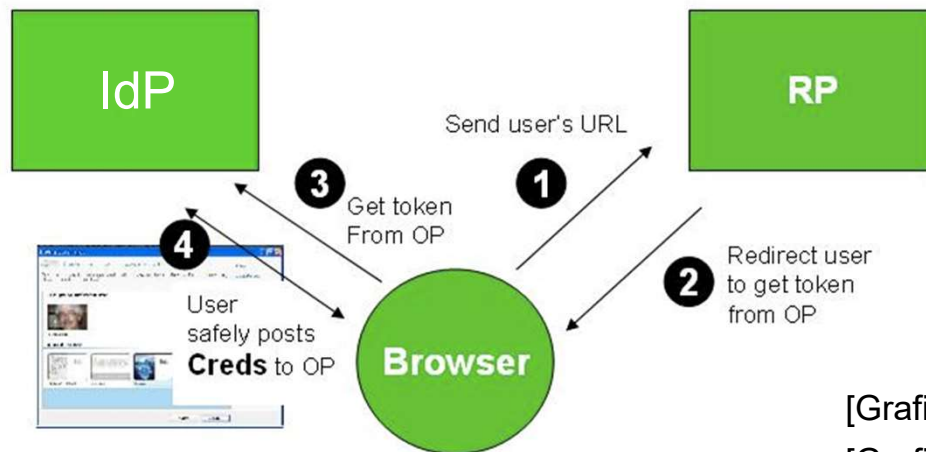
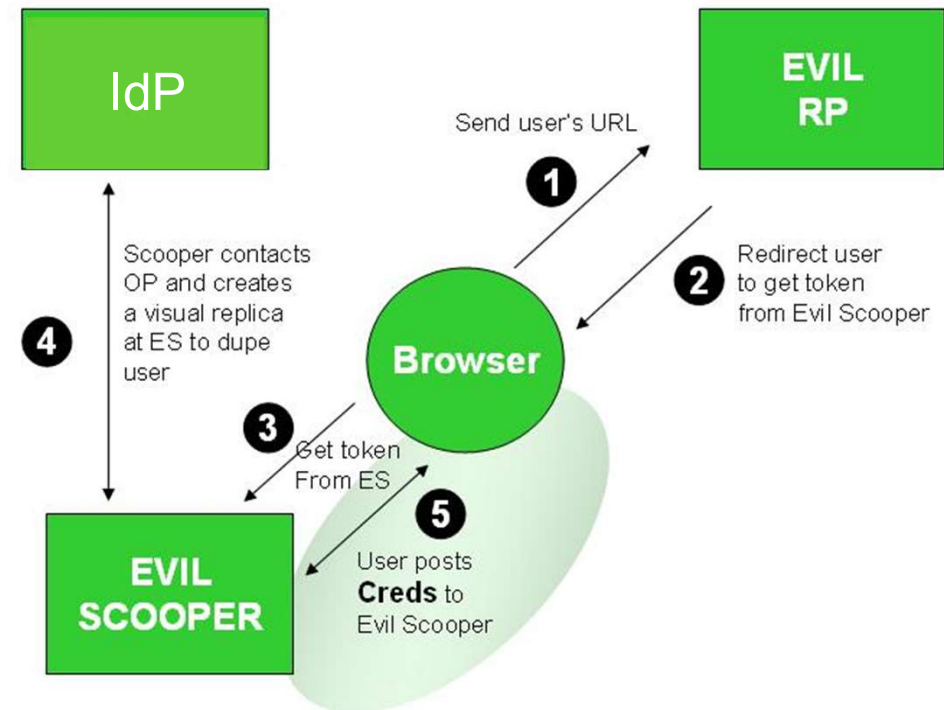
-> u.U. also keine große Erleichterung abgesehen von einheitlichem Login



- heute vielfach:  Login with Facebook  Login with Google
- mittels OpenID Connect (OIDC), basierend auf OAuth 2.0 (Open Authorization, RFC6749)

OpenID – Phishing-Problematik

- Angriff:
 - Angreifer betreibt Service Provider und Fake-IdP (Evil Scooper), der IdP maskiert
 - Fake-IdP phisht Benutzer-Passwort ab
- Abhilfe:
 - Zertifikats-basierte Authentifikation am IdP



[Grafik: <http://www.identityblog.com/wp-content/images/2007/01/openid-2.jpg>]

[Grafik: <http://www.identityblog.com/wp-content/images/2007/01/openid-3.jpg>]

OpenID Connect (OIDC)-Ablauf Demo: <https://www.openidconnect.net/>

1. Bei Klick auf „Login“-Button in Web-App: **Redirect zu OpenID Provider (OP)**

```
GET https://samples.auth0.com/authorize?  
client_id= kbyuFDIdLLm280LIwVFiazOqjO3ty8KH  
&redirect_uri= https://openidconnect.net/callback  
&scope=openid profile email phone address  
&response_type=code  
&state=dfbe7fa3dfc52184662131f186c2ef90b63e89cd
```

- Kennung der Relying Party
- angefragte Attribute
- „Authorization Code“ Flow: Antwort nur kurzlebiger Access Code
- Zustand der Relying Party, wird vom OP zurückgeschickt, Schutz gg. CSRF

2. nach **Login am OP** wird **Authorization Code** an Browser zurückgegeben, der ihn per `redirect_uri` an Web-App (als relying party) weitergibt

3. diesen **tauscht Web-App per Backchannel** (= Server-to-Server) um

```
POST https://samples.auth0.com/oauth/token  
grant_type=authorization_code  
&client_id=kbyuFDIdLLm280LIwVFiazOqjO3ty8KH  
&client_secret=60Op4HFM0I8ajz0WdiStAbziZ-VFQ...  
&redirect_uri=https://openidconnect.net/callback  
&code=Hvhb_Kf2PSgzko2M130cz_xHkFhLn-T9ZnY1ofDFNeV0u
```

- `client_id` und `client_secret` werden bei Registrierung der Web-App am OP festgelegt
- zur AuthN Web-App am OP => muss geheim bleiben (Alternativen: mTLS, signierter Request per JWT)

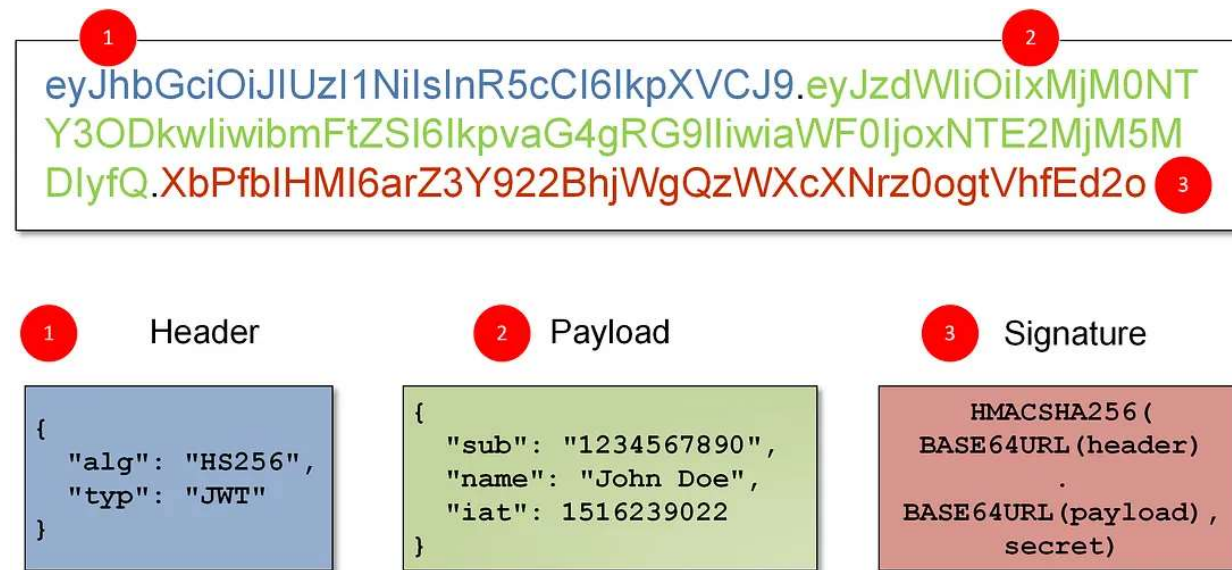
OIDC-Tokens, JWT

vom OP zurückgelieferte Tokens:

- **Identity Token:** weist Identität des Users gg.über Web-App nach, kurze Gültigkeit (ca. 5-30 Min.), Format: immer JSON Web Token für OIDC erforderliche Claims:

issuer, **subject**, **audience**, **expiration time**, **iat** (issued at)

- **Access Token:** für Zugriff auf APIs, ebenfalls kurze Gültigkeit
Nutzung i.d.R. per Header `Authorization: Bearer <access-token>`
Format: JWT oder „opaque“ (zufälliger String, dann aber Anfrage von Web-App an Token Introspection Endpoint des Authorization Servers erforderlich)
- **Refresh Token:** für Erneuerung von Access und ID Tokens ohne erneuten Login, deutlich längere Gültigkeit (Tage / Wochen)
Format: opaque
OAuth2 [RFC9700] empfiehlt „rotation“, d.h. nach jeder Verwendung wird neues Refresh Token ausgestellt, mehrfache Verwendung deutet auf Kompromittierung hin



ID Token aus Beispiel

Verknüpfung mit mehreren Identitätsquellen möglich, zB.

- Social Provider (Facebook, Google, LinkedIn)
- Enterprise-IdP

Screenshot: <https://www.jwt.io>

```
{
  "user_id": "auth0|6a229dc972621e4b02068732",
  "clientID": "kbyuFDIdLLm280LIwVFiaz0qj03ty8KH",
  "created_at": "2026-06-05T09:58:33.531Z",
  "email": "oidc@byom.de",
  "email_verified": false,
  "identities": [
    {
      "connection": "Username-Password-Authentication",
      "provider": "auth0",
      "user_id": "6a229dc972621e4b02068732",
      "isSocial": false
    }
  ],
  "name": "oidc@byom.de",
  "nickname": "oidc",
  "picture": "https://s.gravatar.com/avatar/18328e33daccfd05eb42",
  "updated_at": "2026-06-07T14:08:28.109Z",
  "user_metadata": {},
  "app_metadata": {},
  "iss": "https://samples.auth0.com/",
  "sub": "auth0|6a229dc972621e4b02068732",
  "aud": "kbyuFDIdLLm280LIwVFiaz0qj03ty8KH",
  "iat": 1780841372,
  "exp": 1780877372
}
```